MINISTRY OF SCIENCE AND HIGHER EDUCATION OF THE RUSSIAN FEDERATION
Federal State Budgetary Educational Institution of Higher Education
"CHELYABINSK STATE UNIVERSITY"
(CSU)
Institute of Information Technology

THESIS

Development of a University Navigator

Authors:
Chikishev Mikhail, PR-102
Mikhail Sharmanov, PR-103
Andreev Alexey, PR-103
Egor Yemelyanov, PR-103

Chelyabinsk
2026

CONTENTS

1 Introduction **3**
1.1 Thesis Rationale 3
1.2 Object and subject 3

2 Theoretical Framework **4**
2.1 Analysis of Current Challenges 4
2.2 Tasks 4
2.2.1 Theoretical Tasks (Subject of the Research) 4
2.2.2 Practical Tasks (Object of the Research) 4
2.3 Study of Technical Materials for Backend Architecture Design . . . 4
2.4 Core Development Principles 5
2.5 Panoramic Content Visualization Methods 5
2.6 Routing Fundamentals 6
2.7 Distributed Communication Protocols 6
2.8 Information Security 7

3 Practical Implementation **8**
3.1 General Architecture Overview 8
3.1.1 Infrastructure Characteristics and Design Principles 9
3.1.2 Technology Stack Components 9
3.2 Backend Implementation 10
3.2.1 Component Initialization and Deployment Process 10
3.2.2 Transport Layer and Request Processing 10
3.2.3 Abstraction Layers: Services and Repositories 10
3.2.4 Background Tasks and Asynchronous Processing 11
3.3 DevOps Methodology and Information Security 11
3.3.1 Continuous Integration (CI) Pipeline and Quality Assurance 11
3.3.2 Containerization and Runtime Security 11

4 Conclusion **13**

1 INTRODUCTION

1.1 Thesis Rationale

The thesis aims to design backend architecture for a replacement of existing navigation service map.csu.ru. The primary driver for the thesis is the need to implement modern 360-degree panoramic imaging technologies to enhance the visualization of the university environment. During the conceptual discussion, it was decided to discontinue support for the current codebase in favor of designing and developing a fundamentally new application. This decision is based on the following factors:

1. low cost-effectiveness of maintaining the previous version due to accumulated technical debt and architectural complexity;
2. the necessity of implementing a modern technology stack to ensure scalability and performance.

The thesis objective is to design a backend of multifunctional ecosystem that addresses key university digitalization challenges and improves the Quality of Life (QoL) for students and staff.

1.2 Object and subject

The subject of the thesis is the active team communication, the study of materials required for the development of the backend architecture, the analysis of existing technical solutions, and the collective decision-making process undertaken during the system design.

The object of the thesis is the set of application and backend flowcharts, along with the description of their interaction.

2 THEORETICAL FRAMEWORK

2.1 Analysis of Current Challenges

The academic process is currently associated with several organizational complexities. These include the absence of a unified secure environment for real-time communication between students and faculty, the decentralization of university information resources, and the low efficiency of notification channels.

The architecture of the project under development is designed to aggregate the necessary services within a single interface, thereby enhancing the transparency and accessibility of the educational environment.

2.2 Tasks

2.2.1 Theoretical Tasks (Subject of the Research)

- facilitate active team communication to discuss the desired system functionalities;
- conduct a collaborative analysis and discuss possible methods for implementing each functionality;
- review the scientific and technical materials required for designing the backend architecture.

2.2.2 Practical Tasks (Object of the Research)

- develop a high-level application flowchart illustrating the interaction among all modules;
- create a detailed backend flowchart.

2.3 Study of Technical Materials for Backend Architecture Design

As part of the theoretical subject of this thesis, a systematic study of technical materials was conducted to inform the creation of the application and backend flowcharts (the object of the research). The study covered four essential areas.

First, the Go programming language was examined using official documentation [**go.dev**], the MetaNIT tutorials [**metanit.go**; **metanit.web**], and

a Stepik course [**stepik.go**]. These sources provided foundational knowledge of Go's concurrency model, web programming capabilities, and microservices design patterns.

Second, geospatial routing technologies were investigated, focusing on pgRouting [**pgrouting.docs**] and its integration with PostgreSQL, as well as practical implementations using OpenStreetMap data. Articles from Habr [**habr.pgrouting**] provided real-world insights into building routing systems.

Third, supporting infrastructure components were reviewed, including Redis for caching and queuing, Asynq [**asynq**] for distributed task processing, and MinIO for object storage of panoramic images. Key Habr resources [**habr.redis**; **habr.minio**; **habr.asynq**] guided these decisions.

Fourth, security practices were studied, specifically the "Shift Left Security" paradigm and its application to Go microservices.

The synthesis of these materials directly shaped the architectural decisions documented in the application and backend flowcharts, thereby completing the theoretical subject of the thesis.

2.4 Core Development Principles

The key priorities in system design are security, performance, portability, and maintainability. These requirements dictated the choice of the technology stack. Several languages, including Python, JavaScript, and C#, were excluded from consideration as they did not meet the performance criteria or the specific architectural requirements of the thesis.

Furthermore, a significant objective for the project team is to gain hands-on experience with high-performance systems and to master industry-relevant skills.

2.5 Panoramic Content Visualization Methods

To achieve a full sense of presence and indoor navigation, 360-degree panoramic imaging technologies will be employed. The key task in the software processing of such content will be the transformation of spherical data into a flat image and its subsequent re-projection within the user interface.

The primary standard used will be the equirectangular projection, where the coordinates of each point on the sphere will be mapped to the Cartesian

coordinates of the image pixels.

Within the scope of this thesis, 360-degree map visualization will be implemented using the specialized rendering library `three.js`, which will map the equirectangular texture onto the inner surface of a virtual sphere or cube. To optimize client-side performance, a segmented loading (tiling) technique will be applied, avoiding excessive RAM consumption when handling ultra-high-resolution images.

2.6 Routing Fundamentals

The system will be based on a mathematical model of a weighted directed graph, constructed according to the principle of Poincaré duality. In this model, 3D indoor volumes will be transformed into nodes, while the connecting surfaces will become edges. The integration of the IndoorGML standard will allow for dynamic network filtering via the `pgRouting` extension. This will enable the calculation of the shortest path using Dijkstra's algorithm on an up-to-date subgraph that accounts for the physical connectivity of floors in real-time.

2.7 Distributed Communication Protocols

To establish a unified interaction environment, the XMPP (Extensible Messaging and Presence Protocol) will be selected. Unlike proprietary solutions, XMPP is based on open XML standards and provides high extensibility through XEP specifications.

The choice of the `ejabberd` server is driven by its fault tolerance, robust API, and ability to handle a large number of concurrent connections. In the application architecture, `ejabberd` will serve as a module providing:

- real-time instant messaging;
- isolation of personal social networks from the learning process;
- multi-user conference management for organizing study groups;
- user notification services.

The protocol integration will ensure a communication channel decoupled from personal data, maintaining traffic and message encryption.

2.8 Information Security

Data protection will be addressed at all stages of the Software Development Life Cycle (SDLC). The choice of the Go programming language is justified by its safety mechanisms (like in KISS). To minimize attack vectors, the following solutions will be implemented:

- use of ORM to prevent SQL injections;
- application of Anubis for protection against automated attacks (bots);
- implementation of specialized Middleware;
- use of JWT for user authorization;
- deployment in "distroless" containers to reduce the attack surface;
- integration of static analyzers and Quality Gates into the CI/CD process.

This approach will implement the "Shift Left Security" concept, allowing for the identification and remediation of vulnerabilities at the early stages of the development cycle.

3 PRACTICAL IMPLEMENTATION

3.1 General Architecture Overview

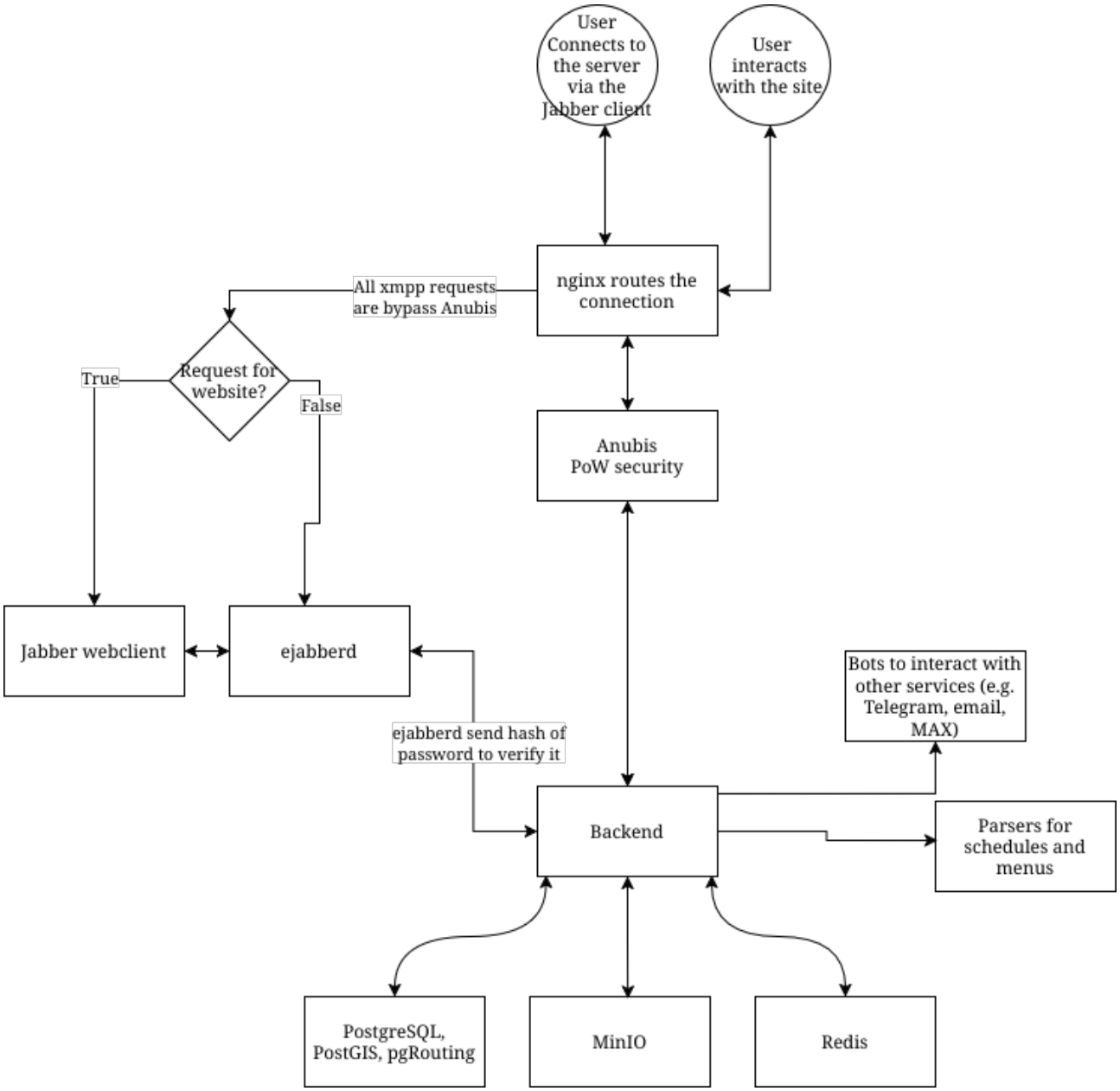

Figure 3–1 – Proposed architecture diagram

The general architecture to be developed is presented in Fig. 3–1. The designed system architecture is dictated by the specifics of the target infrastructure — deployment within an isolated virtual machine (VM) with limited computing resources. These conditions impose several constraints on the use of orchestration tools and necessitate a transition to a hybrid software development model.

3.1.1 Infrastructure Characteristics and Design Principles

The choice of a `docker-compose` based deployment environment limits horizontal scaling capabilities in the short term; however, it ensures high autonomy in system resource managment. The core architectural principles will include:

- Service Consolidation: Minimizing the number of containers to reduce virtualization overhead.
- Hybrid Monolith: Business logic will be implemented within a single service (Backend) to simplify inter-node communication.
- Stateless Approach: The backend will be designed without session state persistence, ensuring readiness for future scaling via Kubernetes.

3.1.2 Technology Stack Components

1. Networking and Security (NGINX, Anubis)
   To handle traffic routing and ensure security, a combination of NGINX and the Anubis protection system will be utilized. This solution will be critical for preventing system performance degradation caused by unauthorized traffic spikes.
2. Storage and Geospatial Analysis Layer (PostgreSQL, PostGIS, pgRouting)
   PostgreSQL will be used as the primary DBMS. The PostGIS extension will provide efficient storage of spatial data, while the pgRouting extension will be integrated to execute shortest-path algorithms directly on the database side.
3. Communication Subsystem (Ejabberd and XMPP Client)
   The `ejabberd` server will be used to facilitate user interaction. User authentication will be delegated to the main backend using the OAuth protocol to enable a Single Sign-On (SSO) concept.
4. Object Storage and Caching (MinIO, Redis)
   Static content will be stored in the `MinIO` object storage. `Redis` will be employed for task queue management and real-time data caching to improve system response times.
5. External Integration Services (Parsers and Bots)
   Data collection modules will be implemented in Python for flexibility. For chat-bot development, priority will be given to the Go language to minimize

resource consumption.

3.2 Backend Implementation

The backend of the application will be developed in Go using a multi-layered architecture. This section examines the application lifecycle and subsystems based on the plan shown in Fig. 3–2.

3.2.1 Component Initialization and Deployment Process

The application startup will follow a strictly deterministic sequence. The deployment algorithm will include:

1. Configuration: Using `Viper` to load environment parameters for flexible setting adjustments.
2. Database State Management: Executing automatic migrations using `golang-mig` to guarantee data schema consistency.
3. Caching and Queue Initialization: Creating clients for `Redis` and `Asynq`.
4. Dependency Injection (DI): Implementing the IoC (Inversion of Control) pattern to facilitate unit testing and eliminate hidden dependencies.
5. Transport Layer Startup: Initializing HTTP server routes and activating background workers.

3.2.2 Transport Layer and Request Processing

The `Gin` framework was selected as the primary HTTP framework. The lifecycle of an incoming request will include passing through a chain of *middleware*, routing to the handler, and invoking business logic via the service layer. To ensure interactivity, a WebSocket Hub subsystem will be implemented.

3.2.3 Abstraction Layers: Services and Repositories

The application architecture is divided into logical levels to isolate business logic from infrastructure implenation details:

1. Service Layer: Will encapsulate the core business logic of the application. Services will operate with DBMS through repository interfaces, remaining independent of specific storage implementations.

2. Data Access Layer (DAL): Will be implemented via the "Repository" pattern to hide the details of SQL queries.
3. Interface Abstraction: The use of interfaces will allow for the replacement of real database connections with *mock* objects during testing.

3.2.4 Background Tasks and Asynchronous Processing

The `Asynq` library will be used to perform resource-intensive operations, such as schedule parsing. Tasks will be placed into a `Redis` queue and processed by dedicated workers to keep the main HTTP server responsive.

3.3 DevOps Methodology and Information Security

A modern CI/CD cycle centered on the "Shift Left Security" concept will be implemented within the thesis. The primary objective will be to identify defects at the earliest possible stages.

3.3.1 Continuous Integration (CI) Pipeline and Quality Assurance

The code build and verification process will be automated using the Jenkins automation server. The lifecycle of changes will include:

1. Automated Testing: Every Pull Request will trigger a suite of unit and integration tests.
2. Static Analysis and Quality Gates: Source code will undergo scanning in SonarQube to assess technical debt and vulnerabilities.
3. Mandatory Code Review: Branch merging will be permitted only upon approval from other members of the development team.

3.3.2 Containerization and Runtime Security

To minimize attack vectors, containerization technology utilizing specialized images will be employed.

- Distroless Containers: The application will use *distroless* images to reduce the "attack surface" by excluding unnecessary OS utilities.
- Multi-stage Builds: The Go application will be compiled in an intermediate container to ensure a minimalist final image.

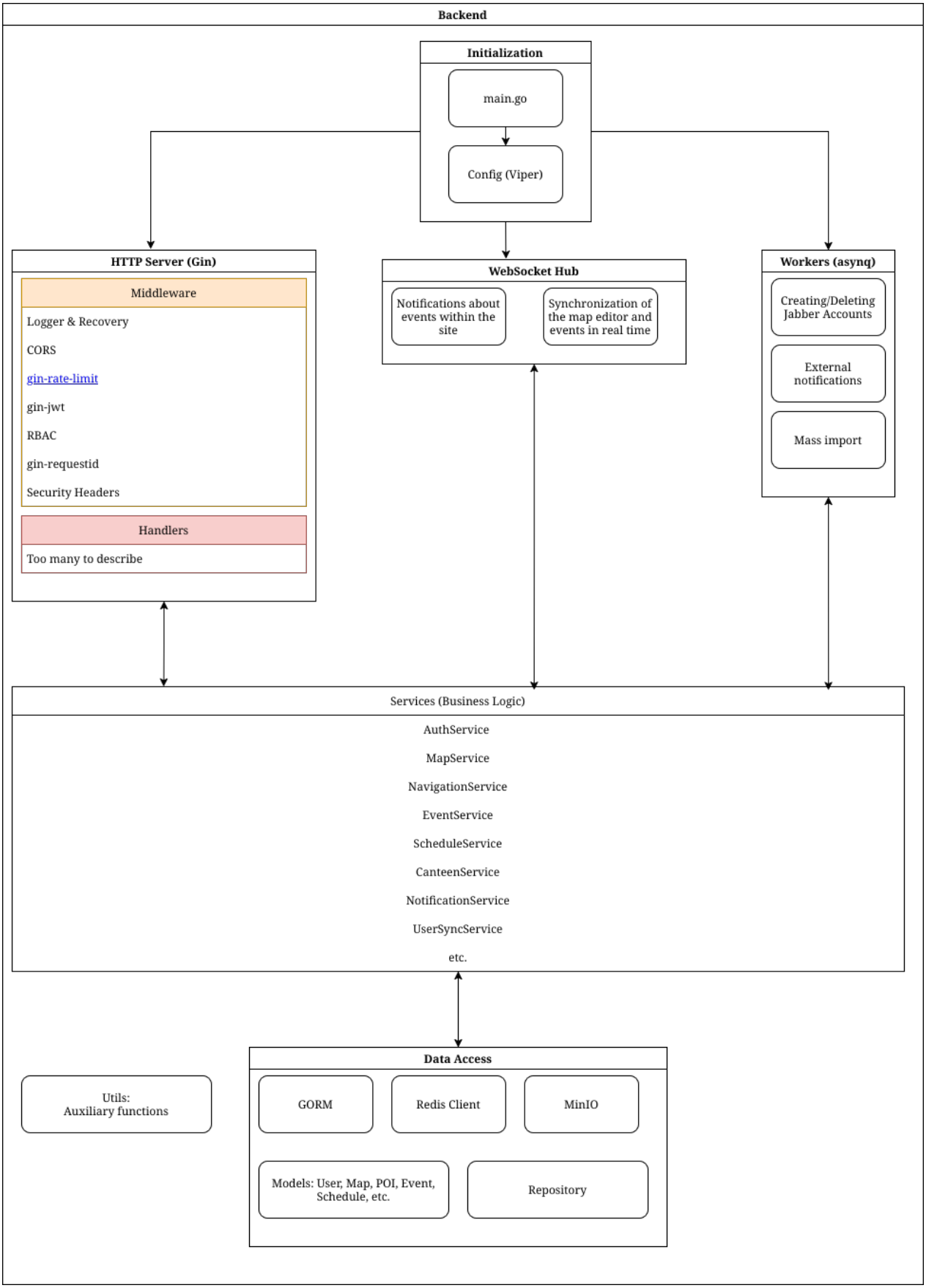

Figure 3–2 – Planned backend block diagram

4 CONCLUSION

In accordance with the research tasks defined in this thesis, the work was structured around two complementary dimensions: the subject and the object.

Regarding the subject of the thesis, all theoretical tasks were successfully completed. Active team communication was facilitated to discuss the desired system functionalities. Collaborative analysis was conducted to evaluate possible implementation methods for each function. Furthermore, the necessary scientific and technical materials required for designing the backend architecture were thoroughly studied.

Regarding the object of the thesis, the practical tasks were fulfilled as follows. A high-level application flowchart illustrating the interaction among all modules was developed. In addition, a detailed backend flowchart, including data flow descriptions and request processing logic, was created. These flowcharts collectively document the proposed architecture of a multifunctional university navigation ecosystem.

The key expected outcomes of the thesis, as reflected in its object, include:

- **Architectural Modernisation:** The designed modular structure, formalised in the flowcharts, provides a foundation for replacing outdated visualisation tools and establishing a unified digital space.
- **Navigation Innovation:** The detailed backend flowchart incorporates provisions for accurate indoor routing (e.g., via integration with frameworks such as pgRouting), while the application flowchart includes support for 360-degree panoramas as a visual aid.
- **Security and Reliability:** The architectural design follows the principles of the "Shift Left Security" concept, and the documented component separation enables the use of minimal (e.g., distroless) containers for deployment.
- **Scalability:** The technology stack indicated in the technical specification (derived from the object of this thesis) allows for seamless project scaling as load increases.

Once implemented based on the designed flowcharts, the system will comply with modern standards and provide a significant improvement in the quality of interaction within the university's digital environment. Thus, the theoretical

subject (team processes and study) enabled the creation of the practical object (application and backend flowcharts), which in turn fully addresses the initial challenges.

BIBLIOGRAPHY

[1] Go Authors. (n.d.). *Go Documentation*. https://go.dev/doc/

[2] MetaNit. (n.d.). *Руководство по языку Go*. https://metanit.com/go/tutorial/

[3] MetaNit. (n.d.). *Руководство по веб-программированию языку Go*. https://metanit.com/go/web/

[4] Stepik. (n.d.). *Course: Development of Web Services in Golang*. https://stepik.org/course/187490/promo

[5] pgRouting Contributors. (n.d.). *pgRouting Documentation (latest)*. https://docs.pgrouting.org/latest/en/index.html

[6] Habr. (2019). *Делаем маршрутизацию (роутинг) на OpenStreetMap. Введение*. https://habr.com/ru/articles/511144/

[7] ManyChat. (2020). *Redis на практических примерах*. https://habr.com/ru/companies/manychat/articles/507136/

[8] OzonTech. (2021). *Зачем и как хранить объекты на примере MinIO*. https://habr.com/ru/companies/ozontech/articles/586024/

[9] Hibiken. (n.d.). *Asynq*. https://github.com/hibiken/asynq

[10] Wunderfund. (2021). *Разбираемся с Redis*. https://habr.com/ru/companies/wunderfund/articles/685894/